

03_consumer_integration_surfaces

HXC-159 Phase 1a — Consumer-side integration surfaces (helix_code + helix_agent)

Field	Value
Revision	1
Created	2026-07-29
Last modified	2026-07-29
Status	Complete — both consumers surveyed read-only; 15 divergences enumerated
Workable item	HXC-159 (Task, Queued, High)
Consumers	helix_code/ (dev.helix.code, go 1.26) · submodules/helix_agent (dev.helix.agent, go 1.26)
Method	Two parallel read-only subagent surveys; every claim carries a file:line

Table of contents

- [1. Headline: do NOT build a skills system — two already exist](#)
 - [2. The hard structural constraint: internal/ visibility](#)
 - [3. helix_code — existing machinery](#)
 - [4. helix_agent — existing machinery](#)
 - [5. Divergences between the two consumers](#)
 - [6. Attachment points, ranked](#)
 - [7. Live defects found during the survey](#)
 - [8. What Phase 2 must decide](#)
 - [9. Honest boundary](#)
-

1. Headline: do NOT build a skills system — two already exist

Both consumers ship a **complete, working, first-class skills subsystem**. Neither is a stub. Under §11.4.74 (extend-don't-reimplement) the incorporation question is therefore *not* “how do we add skills to HelixCode” but “**which of the three existing skill models wins, and how do the other two converge onto it.**”

	helix_code	helix_agent	HelixSkills (upstream)
Package	internal/commands (+ internal/agent)	internal/skills	internal/skill + internal/models
Core type	Skill — all fields unexported , accessor-based	Skill — all fields exported , flat struct	models.Skill — exported, DB/JSON/TOML-tagged
Trigger model	compiled []* regexp.Regexp w/ named captures	[]string phrases scraped from description prose	none — graph edges + semantic search
Precedence	3 tiers (builtin/user/project) + 2 layouts	1 flat directory , silent overwrite	DB rows, no tiers
Execution	template render + substitution	LLM prompt-injection + hardcoded category switch	REST/MCP/CLI serving
Hot reload	reconciling diff	30 s polling ticker (self-admitted stopgap)	fsnotify-gated config flag (unimplemented)
Isolation	requires_isolation first-class	absent	n/a
Persistence	none	none	PostgreSQL + pgvector
Corpus on disk	1 builtin + 1 project skill	1 174 SKILL.md files in skills/	7 in constitution, graph-served

Three key asymmetries fall out immediately:

1. **helix_agent has the corpus** (1 174 SKILL .md across 15 vendor namespaces: azure, claude-code, codex, data, development, devops, forge, github-copilot, gptme, misc, openhands, plugins, postgres-mcp, ui-ux, web) but the **weakest engine**.
2. **helix_code has the best engine** (compiled regex triggers with named captures, three-tier precedence, reconciling reload, isolation flag) but almost **no corpus** (2 skills).
3. **HelixSkills has the only persistence and the only graph** — and neither consumer has *any* skill persistence at all (verified absent in both).

That distribution is the single most useful fact for planning: the three systems are complementary, not redundant.

2. The hard structural constraint: internal/visibility

This constrains every design option and must be settled before anything else.

Dependency direction is one-way and verified in both directions:

- `helix_code/go.mod:6` — `require dev.helix.agent v0.0.0-...`
- `helix_code/go.mod:240` — `replace dev.helix.agent => ../submodules/helix_agent`
- `helix_code/go.mod:251` — `replace dev.helix.agent/pkg/api => ../submodules/helix_agent/pkg/api`
- `helix_code/go.mod:253` — `replace github.com/HelixDevelopment/helix_agent/Toolkit => ../Toolkit`
- **Reverse: zero.** A `grep` for `dev.helix.code` across every `.go/.mod` in `helix_agent` returns no matches — correct per CONST-051(B) decoupling.

helix_agent's entire public Go surface is 4 files:

<code>pkg/api/llm-facade.pb.go</code>	(generated protobuf; own module <code>dev.helix.agent</code>)
<code>pkg/api/llm-facade_grpc.pb.go</code>	(generated gRPC)
<code>pkg/sdk/go/verifier/client.go</code>	(LLMsVerifier HTTP client)
<code>pkg/sdk/go/verifier/client_test.go</code>	

Consequence: `internal/skills`, `internal/plugins`, `internal/tools`, `internal/services` and `internal/agents` in `helix_agent` are **structurally**

unreachable from `dev.helix.code` under Go's `internal/` rule. `helix_code`'s own source acknowledges this in three places:

- `helix_code/internal/llm/ensemble_provider.go:23` — “*internal/packages (dev.helix.code cannot import dev.helix.agent/internal/...)*”
- `helix_code/internal/llm/keyrecognition.go:14` — “*NOT import helix_agent...*”
- `helix_code/cmd/cli/main.go:1550` — “*on the shared substrate when the replace dev.helix.agent wiring lands.*”

The only proven cross-module seam is the `pkg/sdk/go/verifier` pattern, consumed at `helix_code/internal/agentbridge/bridge.go:19` (`agentverifier "dev.helix.agent/pkg/sdk/go/verifier"`). Its own doc comment (`bridge.go:1-11`) states the intent: `HelixCode` depends on the *bridge*, not the foreign module path, “keeping the coupling point in one place.”

Therefore any shared skills contract must be one of exactly three shapes: (a) promoted into `helix_agent/pkg/sdk/go/skills` mirroring the verifier precedent; (b) an over-the-wire contract (HTTP/MCP), coupling nothing at the Go level; (c) a new third module both consumers import — for which a candidate already exists and is mis-wired (see §7.4).

3. `helix_code` — existing machinery

3.1 The skills subsystem (`internal/commands/markdown_skills.go`, 429 lines)

```
// internal/commands/markdown_skills.go:33-42  - ALL FIELDS
    UNEXPORTED
type Skill struct {
    name, description, body string
    variables                map[string]string
    triggerPatterns          []string           // raw (preserved
                        for /skills show)
    triggers                 []*regexp.Regexp  // compiled; bad
                        regex skipped at parse time
    requiresIsolation        bool
    sourcePath               string
}

// :45-50 — the on-disk schema
type skillFrontmatter struct {
    Description string           `yaml:"description"`
    Triggers    []string          `yaml:"triggers"``
```

```

Variables      map[string]string `yaml:"variables"`
RequiresIsolation bool
               `yaml:"requires_isolation"`
}

```

Accessors at :53 Name(), :56 Description(), :59 SourcePath(), :62
 RequiresIsolation(), :65 Body(), :68 TriggerPatterns().

Component	Location	Contract
SkillRegistry	:146	NewSkillRegistry:152, Add:157, Remove:164, Get:171, List:179 (sorted FindMatching(input) (*Skill, map[string]string, bool):198
SkillLoader	:252	NewSkillLoader(reg, projectDir, userDir):263, Load:277, Reload:283, Loaded:413
Precedence	:234-251, impl :283-354	builtin (//go:embed, :26-30) → user → project; project wins
Layouts	:307-322, :326-338	flat <dir>/<name>.md and packaged <dir>/<name>/SKILL.md (skillManifestName:232)
SkillsWatcher	skills_watcher.go:15	fsnotify, 200 ms debounce (:38), SetDebounce:54, Run:65, Close:61
Rendering	:123, :132	Render(args, selection, currentFile), RenderWithCaptures(args, captures ...)
/skills command	skills_command.go:12	list:60, show:73, reload:90, invoke:103 — fully i18n'd
helixcode skills	cmd/cli/skills_cmd.go:22	list:34, show:50, invoke:75, reload:97
SkillDispatcher	internal/agent/skill_dispatcher.go:19	Match:35
N-directory loader	internal/agent/skill_activation.go:23	LoadSkillsAndDispatcher(skillsDir []string) — already generalises to an ordered list (:32-40)

Substitution tokens (shared with markdown slash-commands, resolver at markdown_commands.go:128-183): {{ARG1}}...{{ARGn}}, {{ARG.name}},

{{SELECTION}}, {{CURRENT_FILE}}, {{CWD}}, {{ENV.X}}, {{FILE:path}} (1 MiB cap at :167). Unknown tokens pass through verbatim (:180).

3.2 The tool system — the model-callable seam

```
// helix_code/internal/tools/registry.go:30-69
type Tool interface {
    Name() string
    Description() string
    Execute(ctx context.Context, params map[string]interface{})
        (interface{}, error)
    Schema() ToolSchema
    Category() ToolCategory
    Validate(params map[string]interface{}) error
    RequiresApproval() approval.ApprovalLevel
}
```

ToolRegistry:100, Register:298 (exported, **last-write-wins, no error**), 22 built-ins hardcoded in registerAllTools:248-295, 14 categories at :82-97. Execution pipeline at :636: plan-mode gate (:522) → approval + sandbox markers (:550) → before-hooks (:721) → telemetry → tool → after-hooks (:742) → LSP notify → auto-commit (:445). Exposure to the model via registryToLLMTools (internal/agent/tool_loop.go:470-497).

Precedent for external registration: ask_user is deliberately *not* auto-registered (registry.go:283-287) and is wired from cmd/cli/main.go instead.

3.3 MCP — client and server, with a working bridge into tools

Client Client:lifecycle.go:31 (Connect:102, Tools:63, CallTool:206), multiplexer Manager:registry.go:20 (Tools:53, CallTool:76, Reload:167), 4 transports (stdio/HTTP/SSE/WS), OAuth in oauth.go. Config .helixcode/mcp.yml → Config/ServerSpec (internal/mcp/config.go:15-56), with ReadOnly:37 mapping a server's tools to approval.LevelReadOnly. Server side MCPServer:server.go:19, RegisterTool:188.

Bridge: (*ToolRegistry).RegisterMCPManager(m *mcp.Manager) at internal/tools/registry.go:886, collision-safe naming in registerMCPToolKey:968 (HXC-113). **Documented limitation :881-885: called once at startup, not reconciled after Manager.Reload.**

3.4 Registries (5) and discovery patterns

Registry	Location	Semantics
SkillRegistry		Add replaces silently

Registry	Location	Semantics
	internal/commands/ markdown_skills.go:146	
commands.Registry	internal/commands/ registry.go:12	Register:27 errors on duplicate name or alias
tools.ToolRegistry	internal/tools/ registry.go:100	Register:298 last-write-wins
plugins.Registry	internal/plugins/ registry.go:5	dead code in wiring (\$7.2)
agent.AgentRegistry	internal/agent/ agent.go:217	Register:230

No `init()`-side-effect registration exists anywhere in the module. Every registry is an explicit mutex-guarded map populated from wiring code. Discovery is directory-scan + parse, plus one `//go:embed` (builtin skills) and YAML config for MCP/hooks.

Note: `internal/discovery/` is **service** discovery (ports/health/broadcast) — not skill machinery.

3.5 Skill config: does not exist

`grep -rni "skill" internal/config/ config/` → **zero hits**. Skill directories are hardcoded string literals at four call sites:

Consumer	Project dir	User dir	Location
CLI interactive	<code>./.helix/skills</code>	<code>os.UserConfigDir()/helixcode/skills</code>	<code>cmd/cli/main.go:1043-1047</code>
CLI skills subcmd	<code>./.helix/skills</code>	<code>os.UserConfigDir()/helixcode/skills</code>	<code>cmd/cli/main.go:3287-3291</code>
TUI	<code>.helix/skills</code>	<code>\$HOME/.helix/skills</code>	<code>applications/terminal_ui/main.go:367-368</code>
desktop (clientcore)	<code>.helix/skills</code>	<code>\$HOME/.helix/skills</code>	<code>internal/clientcore/agent.go:142-143</code>

The CLI and the TUI/desktop disagree on the user directory — a live inconsistency (\$7.3).

4. helix_agent — existing machinery

Scale: 3 149 .go files, 727 directories containing Go (excluding vendor/).
internal/ = 1 707 files across 66 subpackages. Two nested modules: pkg/api/
go.mod and Toolkit/go.mod (github.com/HelixDevelopment/helix_agent/
Toolkit, go 1.24.11).

4.1 internal/skills — the primary system (11 non-test files)

Package doc (types.go:1-4): “Skills are structured instruction sets that teach the AI when and how to perform specific tasks, auto-activating based on conversation triggers.”

```
// internal/skills/types.go:12-44 — ALL FIELDS EXPORTED
type Skill struct {
    Name, Description string
    AllowedTools      string    `yaml:"allowed-tools"`
    Version, License, Author, Category string
    Tags, TriggerPhrases []string
    Overview, WhenToUse, Instructions string
    Examples           []SkillExample
    Prerequisites, Outputs string
    ErrorHandling []SkillError
    Resources, RelatedSkills, RawContent, FilePath string
    LoadedAt, UpdatedAt time.Time
}
```

Component	Location	Notes
Parser	parser.go:25	ParseFile:38, Parse:48, ParseDirectory:337 (matches SKILL.MD case-insensitively :345)
trigger extraction	parser.go:32, :18	regex (?i)triggers?\s+on:?\s*([^.]+) over the description prose + quoted-string scraping
Matcher	matcher.go:13	matchExact:98, matchPartial:120, matchFuzzy:150, matchSemantic:181; Match:49, MatchBest:248, MatchMultiple:262

Component	Location	Notes
SemanticMatcher	matcher.go:21-23	clean injection interface — Match(ctx, input, candidates) ([]float64, error)
Registry	registry.go:27-38	4 safe.Store indexes + writeMu; CONST-029 concurrency contract at :16-26
Service	service.go:15	30-method facade
SkillLoader	loader.go:14	parallel, unwired load path (\$7.5)
Tracker	tracker.go:18	usage telemetry, in-memory only
Integration	integration.go:13	ProcessRequest:34, EnhancePromptWithSkills:123 — prompt injection is the execution model
ProtocolSkillAdapter	protocol_adapter.go:36	projects skills as MCP tools / ACP actions / LSP commands

Registry methods: NewRegistry:49, Load:73, LoadFromPath:107, RegisterSkill:125, Get:163, GetByCategory:168, GetByTrigger:179, GetAll:190, GetCategories:195, GetTriggers:200, Remove:205, Stats:254, EnableHotReload:297, DisableHotReload:319, Search:344.

Hot reload is a **polling ticker**, self-admitted at registry.go:336-337: “*Simple implementation: just reload periodically / A more sophisticated version would use fsnotify.*”

Wiring (internal/router/router.go:331-345) — note the failure mode:

```
skillConfig := skills.DefaultSkillConfig()
skillConfig.SkillsDirectory = "skills" // hardcoded
single path
skillService := skills.NewService(skillConfig)
if initErr := skillService.Initialize(context.Background());
    initErr != nil {
    logger.WithError(initErr).Warn("Failed to initialize skills
    system, continuing without skills")
}
```

Skill-system init failure is non-fatal. The server runs on without skills.

HTTP surface (router.go:1359-1370, all auth'd): GET /v1/skills, GET /v1/skills/categories, GET /v1/skills/:category, POST /v1/skills/match. **Read + match only** — no create/update/delete.

Importers of internal/skills (non-test, exhaustive): internal/router/router.go, internal/handlers/{openai_compatible, debate_handler, skills_handler, completion}.go. Nothing else in 727 packages consumes skills.

4.2 Two tool abstractions inside one module

```
// internal/tools/handler.go:15-20
type ToolHandler interface {
    Name() string
    Execute(ctx, args map[string]interface{}) (ToolResult,
        error) // ToolResult
    ValidateArgs(args map[string]interface{}) error
    GenerateDefaultArgs(context string) map[string]interface{}
}

// internal/services/tool_registry.go:14-20
type Tool interface {
    Name() string
    Description() string
    Parameters() map[string]interface{}
    Execute(ctx, params map[string]interface{}) (interface{},
        error) // interface{}
    Source() string
}
```

Different return types for the same concept. **internal/tools** has **ZERO non-test importers** — a fully-built, unwired subsystem.

The cleanest generic hook in either consumer lives in the other one:

```
// internal/services/tool_registry.go:113
func (tr *ToolRegistry) RegisterExternalToolSource(sourceName
    string, toolFetcher func() ([]Tool, error)) error
```

plus RegisterCustomTool(Tool) error:51. **Caveat:** RefreshTools:138 clears all non-"custom"-source tools (:143-147) and re-pulls only MCP+LSP — an external source would be silently dropped on refresh.

Skills are registered into neither tool registry. ProtocolSkillAdapter keeps its own parallel MCPSkillTool/ACPSkillAction/LPSkillCommand maps.

4.3 Plugins — LLM-provider-shaped, not general

```
// internal/plugins/plugin.go:11-30
type LLMPlugin interface {
    Complete(ctx, *models.LLMRequest) (*models.LLMResponse,
        error)
    CompleteStream(ctx, *models.LLMRequest) (<-chan
        *models.LLMResponse, error)
    Name() string; Version() string
    Capabilities() *models.ProviderCapabilities
    Init(config map[string]interface{}) error
    Shutdown(ctx) error; HealthCheck(ctx) error
    SetSecurityContext(*PluginSecurityContext) error
}
```

Native Go .so via plugin.Open(loader.go:23), symbol literal "Plugin" (:29), discovery by .so suffix (discovery.go:60). **A skills plugin cannot implement this without stubbing** — do not attach here. Non-test importers: only plugins/example/plugin.go and internal/adapters/plugins/adapter.go. Not wired into the router.

4.4 Misnomers worth not wasting a cycle on

- internal/services/plugin_system.go contains **zero plugin machinery** — it is HA / load-balancing / circuit-breaking (HighAvailabilityManager:24, LoadBalancer:58, CircuitBreaker:750).
- extensions/ is TypeScript only (extensions/vscode/src/extension.ts). **No Go Extension abstraction exists in either consumer.**

5. Divergences between the two consumers

Cross-consumer parity is a stated requirement, so each of these is a parity risk. 15 found.

#	Dimension	helix_code	helix_agent
D1	Skill struct	unexported fields + accessors (markdown_skills.go:33)	all exported (types.go:12)
D2	Trigger model		prose-scraped substrings (parser.go:32)

#	Dimension	helix_code	helix_agent
		compiled regex + named captures (:37, FindMatching:198)	
D3	Precedence	3 tiers + embed + 2 layouts (:234-251)	1 flat dir, silent overwrite (registry.go:139)
D4	Registry API	Add/List(sorted)/FindMatching	RegisterSkill/GetAll(unordered)/Search+GetByTrigger
D5	Execution	template render w/ editor context	LLM prompt-injection + closed category switch (protocol_adapter.go:309-579)
D6	Isolation	requires_isolation first-class (:41,:62)	absent ; AllowedTools parsed but never enforced
D7	Hot reload	reconciling diff (:279-282)	30 s poll + full clear (registry.go:336)
D8	Plugin iface	Tools()/Hooks() (types.go:5-12)	LLM-shaped (plugin.go:11)
D9	Plugin manifest	full YAML Manifest (types.go:14-25)	none — .so suffix + symbol lookup
D10	Registry signatures	Unregister(name) no error; List() []Plugin; GetToolPlugins	Unregister(name) error; List() []string
D11	Tool iface	7-method w/ Schema+RequiresApproval	two ifaces, neither matching
D12	Logging	go.uber.org/zap (:262,:274)	github.com/sirupsen/logrus
D13	Type duplication	1 skill type	3 (internal/skills, codexskills:30, claudeplugins:30)
D14	skill_registry module	no reference	dangling replace (\$7.4)
D15	Persistence	none	

#	Dimension	helix_code	helix_agent
			none (15 repositories, no skill repo)

5.1 D2 is the blocking divergence — the two SKILL.md schemas are mutually unreadable

helix_agent corpus, skills/codex/skill-creator/SKILL.md:1-6:

```

---
name: skill-creator
description: Guide for creating effective skills. This skill
            should be used when...
metadata:
  short-description: Create or update a skill
---
```

helix_code builtin, helix_code/internal/commands/builtin_skills/conventional-commit/SKILL.md:1-9:

```

---
description: Draft a Conventional Commits message from a short
            summary of the change
triggers:
  - "(?i)^commit message for (?P<summary>.+)$"
  - "(?i)^write a (?:(conventional )?commit(?: message)? for (?P<summary>.+))$"
requires_isolation: false
variables:
  spec_url: "https://www.conventionalcommits.org/en/v1.0.0/"
---
```

Neither parser produces a usable skill from the other's file. helix_agent has name/allowed-tools/version/license/author/metadata; helix_code has triggers/variables/requires_isolation and derives the name from filename/dirname. And a **third** schema exists upstream (HelixSkills TOML, models.TOMLSkillWrapper).

Three incompatible skill schemas is the central technical problem of HXC-159. No amount of plumbing hides it; Phase 2 must pick a canonical schema and author converters both ways with round-trip tests.

6. Attachment points, ranked

Cross-consumer viability is scored against §2 (internal/ visibility).

Rank	Attachment point	Location
1	<code>agent.LoadSkillsAndDispatcher(skillsDirs []string)</code>	<code>helix_code/</code> <code>agent/</code> <code>skill_activ</code>
2	<code>services.ToolRegistry.RegisterExternalToolSource(name, fetcher)</code>	<code>helix_agent</code> <code>services/</code> <code>tool_regist</code>
3	<code>commands.SkillRegistry + SkillLoader</code>	<code>helix_code/</code> <code>commands/</code> <code>markdown_sk</code>
4	<code>pkg/sdk/go/skills</code> (new, mirroring the verifier)	pattern at <code>hel</code> <code>pkg/sdk/go/</code> <code>client.go + h</code> <code>internal/ag</code> <code>bridge.go:1</code>
5	<code>tools.ToolRegistry.Register(Tool)</code>	

Rank	Attachment point	Location
		helix_code/ tools/regist
6	skills.Service construction site	helix_agent router/ router.go:3
7	skills.SemanticMatcher	helix_agent skills/ matcher.go:
8	mcp.Manager + RegisterMCPManager	helix_code/ mcp/registry tools/regist

Deliberately excluded: helix_agent/internal/plugins.LLMPlugin (plugin.go:11) — LLM-provider- shaped; a skills plugin cannot implement it without stubbing Complete/CompleteStream/ Capabilities.

7. Live defects found during the survey

Each is independently actionable and worth its own tracked item.

7.1 helix_code: no exported Skill constructor

grep "func NewSkill\b" → **zero hits** module-wide. The only constructors are unexported parseSkillFile (markdown_skills.go:78) and

ParseSkillForTest (:427, documented test-only). **You cannot construct a *Skill from outside package commands** except by writing a file to disk. This blocks attachment point #3 and is the single most important structural fact for HXC-159.

7.2 helix_code: skill auto-trigger is dead in the interactive CLI

```
cmd/cli/main.go:1060: _ = agent.NewSkillDispatcher(skillReg,  
nil) // wired into baseAgent in a follow-up
```

The dispatcher is constructed and **thrown away**. Auto-triggering works in the TUI (applications/terminal_ui/main.go:1729-1734) and desktop (applications/desktop/main.go:444) but **not** in the CLI, where skills are reachable only via explicit /skills invoke.

7.3 helix_code: CLI and TUI disagree on the user skills directory

os.UserConfigDir()/helixcode/skills (cmd/cli/main.go:1044, :3288) vs \$HOME/.helix/skills (applications/terminal_ui/main.go:368, internal/clientcore/agentcore.go:143). A skill installed for one surface is invisible to the other.

7.4 helix_agent: dangling + mismatched skill_registry wiring

- submodules/helix_agent/go.mod:279 — replace digital.vasic.skillregistry => ../skill_registry
- **No require line for digital.vasic.skillregistry anywhere in that file.**
- submodules/skill_registry/go.mod:1 declares module dev.helix.agent/skillregistry — a **different path** from the replace key.
- helix_code/go.mod has no skillregistry line at all.

A dedicated skill-registry module exists in the workspace, is wired to neither consumer, and its module path does not match the key that replaces it. Given §2, **this may be the intended third-module answer** — Phase 2 should investigate before designing around it. (Root .gitmodules confirms it: dependencies/vasic-digital/skill_registry → path = submodules/skill_registry, url = git@github.com:vasic-digital/SkillRegistry.git.)

7.5 helix_agent: two parallel skill load paths, one unwired

Registry.Load (registry.go:73) and SkillLoader.LoadFromDirectory (loader.go:44) both walk for SKILL.md. Only Registry.Load is wired;

SkillLoader (with LoadFromConfig:99, LoadBuiltinSkills:148, ReloadSkill:193, GetInventory:241) is dead code.

7.6 helix_agent: AllowedTools is a security no-op

ParseAllowedTools (types.go:173) parses "Read, Write, Bash(cmd:*)" into structured constraints. **No non-test caller exists.** Skills declare tool permissions that nothing enforces.

7.7 helix_agent: malformed skills vanish silently

parser.go:347-351 — parse failures inside ParseDirectory are logged at Debug and skipped. With 1 174 corpus files, a malformed skill disappears with no operator-visible signal.

7.8 helix_code: plugins.Registry is dead code

No production caller of plugins.NewRegistry outside internal/plugins/. MaybeRunPlugin (activation.go:43) consults Loader, not Registry. Also Manifest.Entrypoint, .Sandbox and .Env are parsed and validated but **never read** — ExecutePlugin hardcodes plugins/<name>/main (exec.go:49).

7.9 Both: no skill persistence anywhere

- helix_agent: 15 repositories in internal/database/, **no skill repo**; grep for skill across internal/database/ and internal/storage/ → zero hits. Registry and Tracker (tracker.go:18) are in-memory; all usage telemetry is lost on restart.
- helix_code: internal/persistence/store.go has manager slots for session/memory/focus/template (:98-119) — **none for skills or plugins.**

HelixSkills is the only one of the three with persistence. This is the strongest single argument for **reuse** of the upstream Layer-2 service rather than reimplementation.

7.10 Documentation drift

helix_code/docs/CAPABILITIES.md:150,153 cites terminal_ui/main.go:325 and :1652; the actual current lines are :368 and :1730.

8. What Phase 2 must decide

Ordered by blocking-ness:

1. **Which skill schema is canonical** — `helix_code` regex/variables/isolation, `helix_agent` name/allowed-tools/metadata, or HelixSkills TOML? Converters + round-trip tests either way (§5.1).
 2. **Which layer of the three-layer upstream architecture is in scope** (see 02 §2) — Layer-1 registration, Layer-2 service consumption, Layer-3 catalogue generation, or all three.
 3. **How the shared contract crosses the module boundary** — `pkg/sdk/go/skills`, over-the-wire, or the existing-but-mis-wired submodules/`skill_registry` (§7.4). §2 admits no fourth option.
 4. **Direction of the `helix_agent` edge** — upstream's own `helix-deps.yaml` declares `helix_agent` as a *dependency of* HelixSkills (02 §4.2). Incorporating HelixSkills *into* `helix_agent` inverts that and risks a cycle.
 5. **Whether to fix §7.1/§7.2 first** — both are small, both block or degrade any `helix_code` skills work, and both are independently valuable.
 6. **Whether `helix_agent`'s 1 174-file corpus migrates** to the canonical schema, and if so how the 15 vendor namespaces map onto the graph's domain/kind taxonomy.
-

9. Honest boundary

Verified: every `file:line` in this document was cited by a survey that read the file. Both surveys were read-only — no file was written, no git state mutated, and the four other live agent streams (`internal/server/`, `applications/` + `internal/fyneui/`, `submodules/debate_orchestrator`, and a research sibling) were untouched.

Not verified — **nothing was compiled or run.** Every “works” claim is a code-reading claim, not runtime evidence. Under §11.4.108 these are SOURCE-layer findings only.

Partial coverage, explicitly on the record: - `helix_agent` is 3 149 Go files; `internal/services/` alone is 183 files of which 3 were read. The agent-runtime characterisation in §4 is **partial** — `internal/agents/{dream,kairos,swarm,voice,yolo}/` and `internal/agents/subagent/` were listed, not read. - Toolkit/ (own module, 50 Go files) was **not** surveyed. Whether it carries a competing skill or tool abstraction is **unknown**. - The 693 Go files under `MCP/` and 21 dirs under `mcp-servers/` were not examined. - `helix_agent/internal/plugins/{config,dependencies,health,hot_reload,lifecycle,metrics,reload,`

security, versioning, watcher}.go were listed, not read — including security.go, which defines PluginSecurityContext referenced by code that *was* read. - **CONST-040 gap:** the mandate requires Skills and Plugins capability flags to be sourced from the verifier VerificationResult. In helix_agent/pkg/sdk/go/verifier/client.go only CapabilityScore float64 (:204) was found; per-capability Skills/Plugins booleans could **not** be confirmed. Separately, helix_code/internal/verifier/embedded_server.go:153 states in a code comment that **no runtime probe for Skills/Plugins exists**. Both are flagged, neither diagnosed. - submodules/plugins/ (own go.mod) is outside the inner module and was deliberately not surveyed. - All importer/usage sweeps excluded _test.go, so “no non-test caller” claims mean exactly that and do not imply absence of test coverage.